3.6. Современные технологии разработки программного обеспечения

3. Сети, базы данных, безопасность кода

В этом уроке

- Узнаем как ведётся разработка ПО в современных командах
- Узнаем как работает Git: init, add, commit, push
- Разберёмся как использовать Docker: run, образы, контейнеры
- Узнаем зачем это нужно в ИБ и разработке

Как выглядит современная разработка ПО?

- Разработка = не только код
- Важно:
  - 👥 Командная работа
  - 📦 Безопасность и контроль версий
  - ⚙️ Повторяемость среды
- Сегодня рассмотрим:
  - Git — контроль версий
  - Docker — виртуальная среда для приложений

Автоматизация непрерывной доставки

(Continuous Delivery) — непрерывная доставка. На этой стадии происходит автоматическое развертывание на стенды и тестовые окружения.

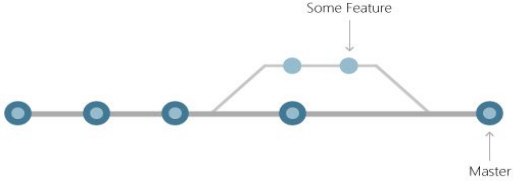
Система контроля версий

Система управления версиями (Version Control System) — программное обеспечение для облегчения работы с изменяющейся информацией.

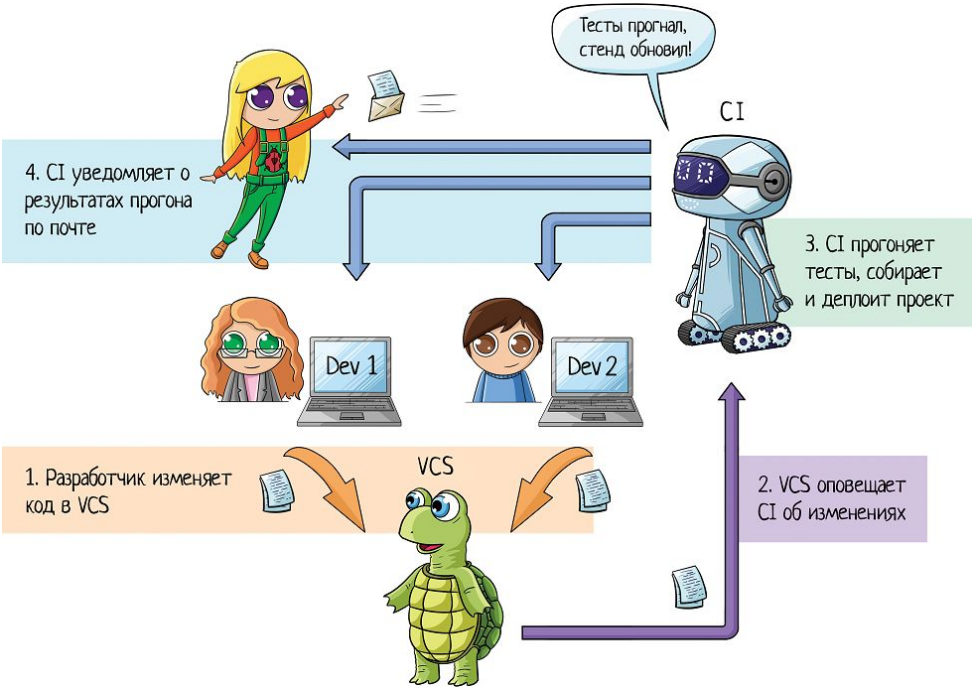
Такие системы наиболее широко используются при разработке программного обеспечения для хранения исходных кодов разрабатываемой программы.

Git — это распределенная система управления версиями, то есть локальный клон проекта — это полный репозиторий управления версиями.

Каждый разработчик сохраняет изменения в собственном локальном репозитории кода. В результате может быть много разных изменений, основанных на одной фиксации. Git предоставляет средства для изоляции изменений и последующего их слияния.



Автоматизация непрерывной интеграции



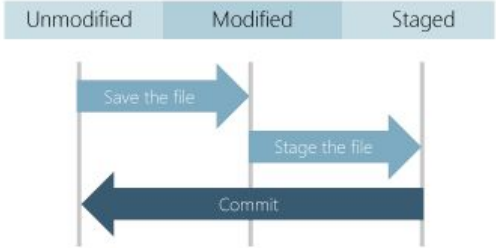
https://habr.com/ru/articles/508216/

Что такое git?

- Система контроля версий
- Позволяет:
  - Сохранять историю изменений
  - Вернуться к любой версии
  - Работать над проектом в команде
  - Видеть, кто что изменил
- 📦 Примеры использования:
- Храним код Python-скриптов
- Ведём историю ИБ-отчётов

Система контроля версий

Файлы в Git находятся в одном из трех состояний: измененных, промежуточных или зафиксированных.
При первом изменении файла изменения существуют только в рабочем каталоге.
Они еще не являются частью фиксации или истории разработки. Разработчик должен *подготовить* измененные файлы для включения в фиксацию. Промежуточная область содержит все изменения, которые необходимо включить в следующую фиксацию. Когда разработчик доволен промежуточными файлами, файлы упаковываются в виде *фиксации* с сообщением, описывающим изменения. Эта фиксация становится частью истории разработки.



Установка Git:
Windows:
https://git-scm.com/download/win
После установки Git доступен из командной строки или PowerShell.
Рекомендуется выбрать значения по умолчанию во время установки.

macOS:
1. Устанавливаем Homebrew
https://brew.sh/
2. Вводим команду $ brew install git

Linux:
1. Инструкция для разных дистрибутивов:
https://git-scm.com/download/linux

Установка и инициализация

```
sudo apt install git      # Linux
git config --global user.name "Имя"
git config --global user.email "почта@example.com"
git init
```

Основные команды Git

```
git status          # Проверить изменения
git add файл.py     # Добавить файл в индекс
git commit -m "Описание"   # Зафиксировать изменения
git log             # Посмотреть историю
```

🔁 Повторение:

- `add` — подготовка
- `commit` — сохранение

Работа с удалённым репозиторием

```
git remote add origin https://github.com/user/project.git
git push -u origin main
```

Git. Начало

Действие	Команды
Настроить свою учетную запись	$ git config --global user.name "My Name" $ git config --global user.email myEmail@example.com
В папке, в которой будем делать изменения запускаем git	$ git init
Добавляем файл в отслеживание	$ git add name.py
Добавить все файлы директории	$ git add -A
Добавить все изменения	$ git add .
Сохраняем изменения и добавляем комментарий	$ git commit –m “Comment”
Проверяем статус файлов	$ git status
Посмотреть изменения	$ git log
Посмотреть разницу между файлом и в git’е	$ git diff
Подключиться к репозиторию	$ git remote add origin https://gitlab.com/your_acc/your_rep.git
Подключиться к основной ветке	$ git branch -M main
Залить файлы	$ git push -uf origin main

Что такое Docker?

- Программа, которая **изолирует среду** запуска
- Позволяет:
  - Упаковать приложение со всеми зависимостями
  - Запускать **одинаково на любом компьютере**
- Очень полезно для:
  - ИБ-анализа

Автоматизированной проверки

Виртуализация

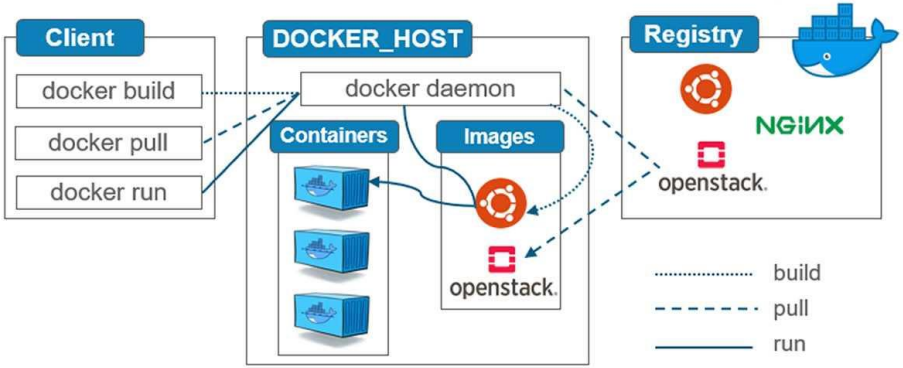
Виртуализация - предоставление набора вычислительных ресурсов или их логического объединения, абстрагированное от аппаратной реализации, и обеспечивающее при этом логическую изоляцию друг от друга вычислительных процессов, выполняемых на одном физическом ресурсе

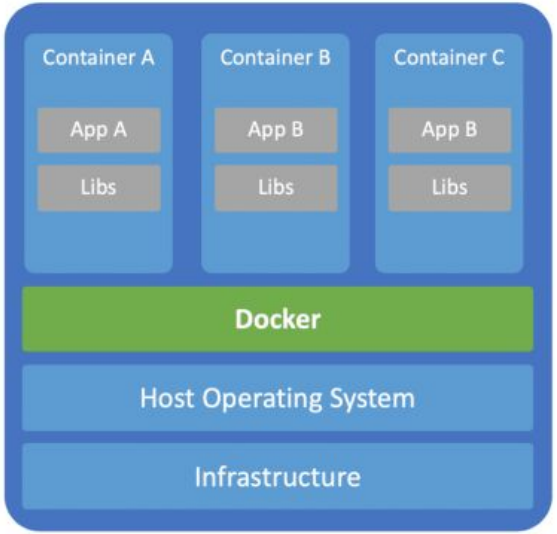
Контейнеризация

Контейнеризация (виртуализация на уровне ОС) – технология, которая позволяет запускать программное обеспечение в изолированных на уровне операционной системы пространствах. Контейнеры являются наиболее распространенной формой виртуализации на уровне ОС. С помощью контейнеров можно запустить несколько приложений на одном сервере (хостовой машине), изолируя их друг от друга.

Docker

Docker – это открытая платформа для разработки, доставки и запуска приложений.





https://www.docker.com/
https://www.hub.docker.com/

Образы и контейнеры

Образ (image) — шаблон окружения

Контейнер (container) — запущенный экземпляр образа

📌 Образы ≈ шаблоны

📌 Контейнеры ≈ работающие программы

Пример запуска Python в контейнер

docker run -it python

📌 -it — интерактивный режим
Теперь можно писать код прямо внутри Docker

Создание своего Dockerfile

```
FROM python:3.11
COPY . /app
WORKDIR /app
CMD ["python", "main.py"]
```

🎁 Создание образа:

```
docker build -t myapp .
docker run myapp
```

Зачем Git и Docker специалисту по ИБ?

Git:

- Хранение скриптов и логов изменений
- Совместная работа над анализаторами

Docker:

- Изолированный запуск **вредоносного** кода
- Тестирование ИБ-сценариев без вреда
- Повторяемые тесты сканеров/парсеров

Итоги урока

- Как работает Git: init, add, commit, push
- Как использовать Docker: run, образы, контейнеры
- Зачем это нужно в ИБ и разработке
- Получили первые навыки работы с Git и Docker